

Topic 1: Introduction to Data Structures and Algorithm Analysis

1.1 What is a Data Structure?

- A data structure is a way of organising, storing, and managing data so that it can be accessed and modified efficiently
- Choice of data structure directly affects the performance of algorithms
- Examples: arrays, linked lists, stacks, queues, trees, graphs, hash tables
- No single data structure is best for every problem -- pick based on operations needed

1.2 Abstract Data Types (ADT)

- An ADT defines what operations a data structure supports, not how they are implemented
- Example: a Stack ADT supports push, pop, peek, isEmpty -- the internal storage (array or linked list) is an implementation detail
- ADTs enable programming to interfaces -- you can swap implementations without changing calling code

1.3 Algorithm Analysis and Big-O Notation

- Big-O notation describes the growth rate of time or space as input size n increases
- $O(1)$ -- constant: array index access
- $O(\log n)$ -- logarithmic: binary search
- $O(n)$ -- linear: sequential scan
- $O(n \log n)$ -- sorting with divide and conquer: merge sort, heap sort
- $O(n^2)$ -- quadratic: nested loops, bubble sort, selection sort
- $O(2^n)$ -- exponential: naive recursive solutions
- We analyse worst-case complexity unless stated otherwise

1.4 Space Complexity

- Memory used by an algorithm as a function of input size
 - Includes memory for variables, call stack (recursion), and auxiliary structures
 - In-place algorithms use $O(1)$ extra space (insertion sort); merge sort uses $O(n)$ extra space
 - Sometimes trade space for time: hash tables use extra memory for $O(1)$ lookups
-